Curtin University — Discipline of Computing

Unix Systems Programming (COMP2002)

Trimester 2, 2026

Assignment

Due: Sunday, 19 July 2026, 11:59 PM (GMT+8)

Weight: 20% of the unit

1 Academic Integrity

This is an assessable task, and as such there are strict rules. You must not ask for or accept help from anyone else on completing the tasks. You must not show your work at any time to another student enrolled in this unit who might gain unfair advantage from it. These things are considered plagiarism or collusion. To be on the safe side, never release your code to the public.

Do NOT just copy some C source codes from internet resources (including AI tools such as ChatGPT). If you get caught, it will be considered plagiarism. If you put the reference, then that part of the code will not be marked since it is not your work.

Staff can provide assistance in helping you understand the unit material in general, but nobody is allowed to help you solve the specific problems posed in this assignment. The purpose of the assignment is for you to solve them on your own, so that you learn from it. Please see Curtin's Academic Integrity website for information on academic misconduct (which includes plagiarism and collusion).

The unit coordinator may require you to attend quick interview to answer questions about any piece of written work submitted in this unit. Your response(s) may be referred to as evidence in an Academic Misconduct inquiry. In addition, your assignment submission may be analysed by systems to detect plagiarism and/or collusion.

2 Task Details

2.1 Task Summary

Please watch the *Quick Demonstration* video provided on the Blackboard. In summary, these are the tasks you need to complete:

- Perform **File IO** operations on a text file with system calls and retrieve information from it.
- Write a simple ASCII-based game utilising **Terminal IO** and **Random Number Generator (RNG)**.
- Display a proper interface to the terminal.
- Use **fork()** to create processes to handle different components of the game.
- Use **flock()** to ensure synchronisity between processes.

You need to use the system calls **open(), close(), read(), write(), fork(), wait(), flock() and nanosleep()** to achieve the objectives. You will also use some **ANSI escape codes** as part of your assignment (Please watch all of the *General Supplementary Videos*). Most C library functions are **NOT** allowed in this assignment.

Note: Please watch the Supplementary Video on the *General Advice* for this assignment and read the provided *ADVICE.txt* on Blackboard. This provides a comprehensive guidance on how to complete the assignment.

2.2 Prohibited Library Functions

Since you need to use system calls, any closely-related library functions are not allowed. This is the list of **banned** library functions:

- Any family function of printf() and scanf() (*e.g* fprintf(), sprintf(), snprintf(), fscanf(), sscanf(), *etc*...)
- fopen() and fclose()
- fgetc(), fgets(), fputc(), fputs() (basically all library functions related to **file IO**)
- atoi() (function to convert string to integer) (If you need this, write your own function)
- Anything from **<string.h>** such as strlen(), strcpy(), strstr(), strcat(), strcmp(), strncmp(), *etc*. If you need to manipulate string, you need to write your own function.
- Anything from math library **<math.h>**
- sleep() function. (Please use *nanosleep()* instead. You can watch the General Supplementary video about this.)

2.3 Allowed Library Functions

You are allowed to use:

- **malloc(), calloc(), and free()** functions to allocate memories (especially for an array).
- **srand() and rand()** functions for **Random Number Generator**.
- **tcgetattr() and tcsetattr()** functions for **terminal IO**.

2.4 Error/Failure Checking

You need to check for error/failure on the initial **open()** system calls when opening the map file at the beginning of the program (just in case the file does not exist). However, you can assume the other **open()** operation on state file is always successful (because we know the file exists at this stage).

2.5 Command Line Arguments

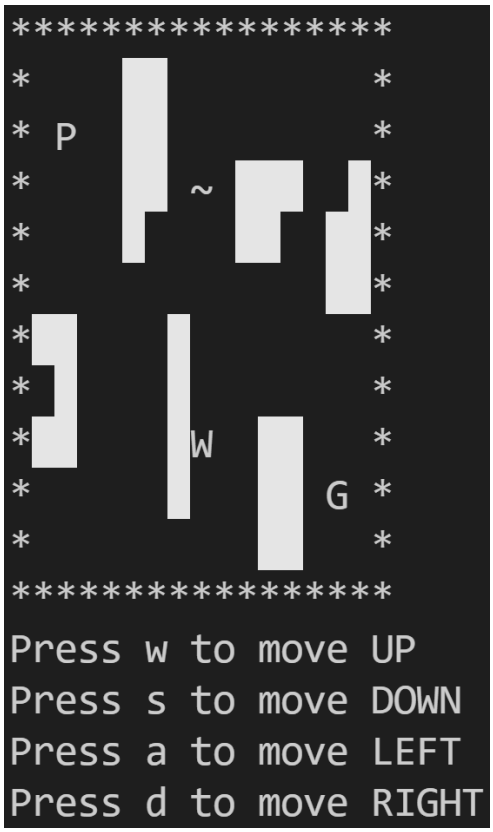
Please ensure that the executable is called **"escape"**. Your executable should accept only one command-line parameter/argument:

./escape <filename>

<filename> is the text file name containing the data you need to extract. It is your responsibility to check whether the file can be opened properly. You can assume the content of the file is valid. If the amount of the command line arguments are too many or too few, your program should print a message on the terminal and end the program accordingly.

2.6 Main Interface

Once you run the program, it should clear the terminal screen (Please watch the relevant *General Supplementary Video*) and print the 2D map containing all the objects: **Walls, Player, Goal, Snake and Wolf**:



Main visual interface of the game.

You can type the command to move the Player. Player and Enemy are not able to go through the Walls and the map borders.

2.7 Characters on the Map

You have to use the correct characters on the game interface:

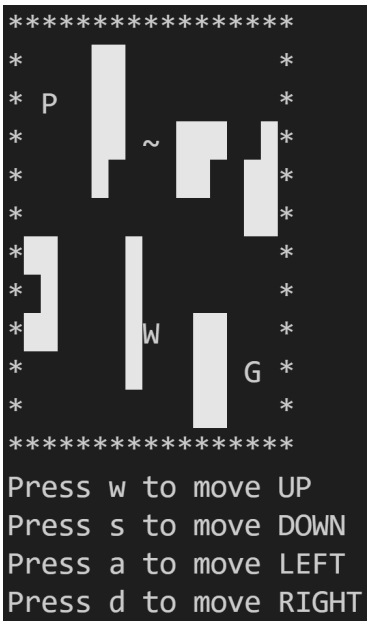
- Char '*****' for the map border.
- An empty space ' ' char for the Wall. (with White background color)
- Char '**P**' for the Player.
- Char '**G**' for the Goal.
- Char '**~**' for the Snake.
- Char '**W**' for the Wolf.

2.8 File I/O

The content of the map file is as follows:

```
0
10 15
0 0 0 0 1 1 0 0 0 0 0 0 0 0 0
0 2 0 0 1 1 0 0 0 0 0 0 0 0 0
0 0 0 0 1 1 0 4 0 1 1 1 0 0 1
0 0 0 0 1 0 0 0 0 1 1 0 0 1 1
0 0 0 0 0 0 0 0 0 0 0 0 0 1 1
1 1 0 0 0 0 1 0 0 0 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0 0 0 0 0 0
1 1 0 0 0 0 1 5 0 0 1 1 0 0 0
0 0 0 0 0 0 1 0 0 0 1 1 0 3 0
0 0 0 0 0 0 0 0 0 0 1 1 0 0 0
```

(a) content of map text file



(b) corresponding interface

Figure 1: This is an example of the map text file and the game interface created from it.

The first integer on the first line indicates the **game state**. '0' means the game is still ongoing, while non-zero means the game is over. The two integers on the second line is the size of the playable map area (**row and column size** respectively). The remaining of the file consists of integers representing the game objects:

- 0 represents ' ' (An empty space)
- 1 represents ' ' (Walls, also an empty space, but with white background)
- 2 represents 'P' (Player)
- 3 represents 'G' (Goal)
- 4 represents '~' (Snake)
- 5 represents 'W' (Wolf)

Note: It is recommended to copy the map file to another text file. This will be very useful when you implement multi-processes feature later on Section 2.11. This serves as a **state file** where all processes communicate to each other via a common file. This **state file** will be updated by all processes for any movement on the map to ensure synchronisity.

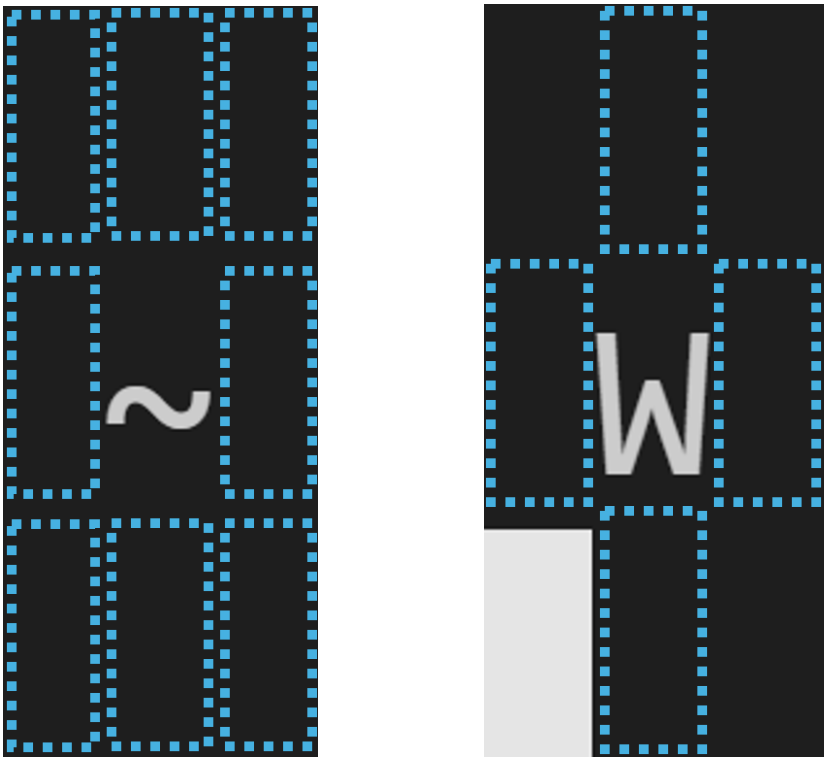
Note: Since you cannot use **fscanf()**, you cannot read the integer easily with **%d** placeholder. You need to use **read()** system call instead. If you need to convert them from string into int, you need to write your own function.

2.9 Enemy Movement Algorithm

Both Snake and Wolf move independently in this game without waiting for the Player movement. They will move after a fixed delay everytime. You can use **nanosleep()** system call for this. Snake will move every 2 seconds, while Wolf moves every second. Snake can move in any of the 8 directions (horizontal, vertical, diagonal), while Wolf only moves horizontally or vertically (4 directions).

Their movement direction is randomly decided. You can use **Random Number Generator (RNG)** for this task. However, when the Player is too close to the Enemy (one move away from being attacked), then the next move would not be random anymore. Instead, the Enemy will attack the Player directly once awaken from the **nanosleep()**. (Attacking means that the Enemy moves to the same location as the Player.)

Note: If the Player get too close to the Enemy, and immediately get away before the Enemy wakes up, then the Enemy still moves randomly as usual.



(a) Snake can move 8 directions. (b) Wolf can only move 4 directions.

Enemy Movement

2.10 Winning and Losing Condition

The game continues until either the Player wins OR loses the game. The **winning condition** is fulfilled when the Player manages to reach the Goal. The **losing condition** is fulfilled when one of the Enemies reach the Player. Both conditions will cease the game, and proceed to clear all the resources (**free()** and **close()**) before terminating the process.

2.11 Multi Processes and Synchronisity

As mentioned in Section 2.9, the Enemies moves independently of the Player. One way to make this possible is to use multi processes with **fork()** system call. **fork()** will split the program into multiple processes. Each process then can handle different objects in the game. For example, we can create two child processes: one will handle Snake movement, while the other handles Wolf movement. Parent process handles the user prompt and Player movement.

For this implementation, you can use the **state file** (from Section 2.8) as the communication media among all processes. Each process opens the state file separately (for BOTH reading and writing purposes). Everytime the Player or Snake or Wolf wants to move, it will use **flock()** system call to get exclusive access to the file until it is unlocked. (Please watch the *General Supplementary Video* about **File IO and fork()**)

When they have the exclusive access, they will read the **state file** to retrieve the latest game state. The game continues if the first integer in the file is still '0'. Once the whole map is retrieved, perform the object movement update on the array and display it to the terminal. Lastly, use **write()** to record the latest game state back into the **state file** (overwriting previous state) before unlocking the file with **flock()**. This ensures other processes always receive the latest game state.

If any movement satisfies the **winning OR losing condition**, that process should update the first integer to be '1' (winning) OR '2' (losing). This is to inform the other processes later on that the game is already over. This will cause all processes to clear all remaining resources (**free()** and **close()**) and terminates its own process. The parent process should use **wait()** system call to ensure other child processes terminate properly.

Note: Depending on your implementation, there could be a lot of **File IO operations** that might affect the performance of the game. As long as it is still reasonably playable, we will accept it. However, if the performance drops significantly (*e.g* one object movement takes one whole second to execute), then you might want to optimise your code.

Note: Another **reminder** to watch the Supplementary Video about *General Advice* and the attached **ADVICE.txt** for this assignment. This provides a more comprehensive guidance to attempt this assignment.

2.12 Makefile

You should write a proper makefile for this assignment. Feel free to use the makefile format learned in **COMP1000 (Unix and C Programming)**, or use the new learned makefile features from this unit. Without the makefile, we will assume the program cannot be compiled. We should be able to compile the assignment in Linux environment.

2.13 Assumptions

For simplification purpose, these are assumptions you can use for this assignment:

- Map file contains valid information to initialise the game.
- All integers in the map file is separated by a single space character. The end of each line is ended with a newline char '\n' (Linux newline, **NOT the Windows newline** "\r\n")
- There is a path from the Player to the Goal. It is always possible to win the game.
- Enemies will not be enclosed within Walls. There is always a chance for the Enemies to reach the Player.
- There is always ONE Player, ONE Goal, ONE Snake, ONE Wolf, and at least ONE Wall.
- The maximum playable map area is 20 rows by 20 columns.

3 Short Report

Please write a short report answering these questions related to the design choice of your program. Your answer should refer to **parts of your code** that achieves the objective.

- How does your program **read()** and parse the integers correctly from the map file to create the 2D array and store the map information? What datatype do you use for the array? Justify your answer. (Remember, you cannot use **fscanf()**)
- How do you use the **Random Number Generator (RNG)** to control the movement of the Snake and Wolf?
- When you use **read()/write()** system calls, do you read/write one char at a time? OR read it as a long string all at once? Justify your answer.
- What is the main challenge from modifying your code from a single-process into the multi-processes game? (There is no wrong answer on this one, but you need to explain your experience when tackling this challenge)

You can submit the report as PDF or TXT file along with your source code.

4 Marking Criteria

This is the marking distribution for your guidance:

- **(5 marks)** Have a proper makefile and it can compile and run the program.
- **(5 marks)** No memory leaks. (You can use **valgrind –trace-children=yes ./escape map.txt**)
- **(5 marks)** Proper error handling on incorrect command line arguments amount, and initial **open()** error checking to ensure the file exist.
- **(10 marks)** FILE IO to retrieve the map file is done correctly. This includes allocating memories and assigning the 2D map array to store the information.
- **(10 marks)** Able to display the game interface and instruction on terminal on a cleared screen. All objects are displayed with the correct characters and background color.
- **(5 marks)** User prompt for Player movement is done correctly with **Terminal IO**. (a single char accepted immediately upon pressing the key)
- **(5 marks)** The Player movement is implemented correctly based on the user prompt. Any invalid move such as moving into the Wall or outside the map border should be detected. Player should not be able to walk into the Enemies as well.
- **(10 marks)** The Snake and Wolf movement are implemented correctly with **Random Number Generator (RNG)**. The Enemies are able to attack the Player when they get too close. Enemies should not be able to walk into Wall and Goal, and they should not be able to move outside the map border.
- **(5 marks)** Winning and Losing condition are implemented correctly. This causes all processes to stop the gameplay loop and properly clear all resources.
- **(30 marks)** Proper implementation of multi-processes game with **fork()** and **flock()**. This allows the Snake and Wolf to move independently of the Player movement. File synchronisation is done correctly among all processes. **nanosleep()** is used to delay the movement of each Enemy. Parent process should use **wait()** system call to wait for all child processes termination. Game Interface should be displayed and updated in real-time for any movement of the Player and Enemies.
- **(10 marks)** A short report containing clear and complete answers, with references to the appropriate sections of the code. A generic answer without reference to your code will not receive mark.

Note: If you use any **PROHIBITED** library functions on specific task, that task will **NOT** receive any mark.

5 Final Check and Submission

After you complete your assignment, please make sure it can be compiled and run on our Linux lab environment. If you do your assignment on other environments (e.g on Windows operating system), then it is your responsibility to ensure that it works on the lab environment. In the case of incompatibility, it will be considered "not working" and some penalties will apply. You have to submit a **single zip file** containing ALL the files in this list:

- **Declaration of Originality Form** – Please fill this form digitally and submit it. This is an important document for assignment submission.
- **Your essential assignment files** – Submit all the .c & .h files and your makefile. Please do not submit the executable and object (.o) files, as we will re-compile them anyway. Do not create any sub-directory within the directory. Every file should exist within the same directory.
- **Brief Report** - Short report answering the questions. Please save it as a PDF or TXT file.

Note: Please compress all the necessary files into a **SINGLE zip file**. So, your submission should only has one zip file. If you do not compress the files, we reserve the right to refuse your submission.

The name of the zipped file should be in the format of:

<your-tutor-name>_<student-ID>_<your-full-name>_Assignment.zip
Example: Antoni_12345678_Will-Smith_Assignment.zip

If you forget your tutor's name, please put the lecturer's name. Please make sure your submission is complete and not corrupted. You can re-download the submission and check if you can compile and run the program again. Corrupted submission will receive instant zero. Late submission will receive zero mark. You can submit it multiple times, but only your latest submission will be marked. Therefore, please make sure the latest submission is the complete code.

Note: If you are granted an extension, please **do NOT submit an early draft** on Blackboard. We might accidentally mark the earlier version of your work. Please **ONLY** submit the final version of your assignment.

End of Assignment